

Object Catch Using Python Turtle Graphics

Theory

The Object Catching Simulation is a simple interactive game built using Python's Turtle Graphics library. The program displays a red circular object at random positions on the screen. The player must click on the object before it disappears. The game tracks successful catches and missed attempts, ending after three misses.

Objectives

- To simulate a reflex-based catching game using mouse input.
- To demonstrate event handling (onclick) and timed events (ontimer) in Turtle Graphics.
- To record player performance through score and miss count.

Materials & Tools

1. Programming Language: Python
2. IDE: Visual Studio Code (VS Code)
3. Library: Turtle Graphics
4. Function: onclick() for mouse interaction and ontimer() for timed events.
5. Output: Turtle Graphics window with interactive interface.

Terminal/console: For print messages; such as

“Caught! Score: X” ,

“Missed! Misses: Y” , and

“Game Over! Final Score: Z” .

Methodology:

- **Environment Setup:** A Turtle screen of size 600×600 pixels is created.
- **Game Object:** A red circle (obj) is initialized but hidden until each round.
- **Game Cycle:**

- `show_object()` places the circle at a random position and makes it visible.
 - After 1.2 seconds, `next_round()` hides the object if not clicked, counts a miss, and starts the next cycle.
 - If clicked (`catch()`), the object is hidden immediately, and the score increases.
- **Termination:** The game ends when the player accumulates 3 misses.

Code Implementation

```
import turtle
import random

screen = turtle.Screen()
screen.title("Click to Catch!")
screen.setup(width=600, height=600)

obj = turtle.Turtle()
obj.shape("circle")
obj.color("red")
obj.penup()
obj.hideturtle()
score = 0
misses = 0

def show_object():
    global misses
    if misses >= 3:
        print("Game Over! Final Score:", score)
        return

    x = random.randint(-250, 250)
    y = random.randint(-250, 250)
    obj.goto(x, y)
    obj.showturtle()
    screen.ontimer(next_round, 1200)

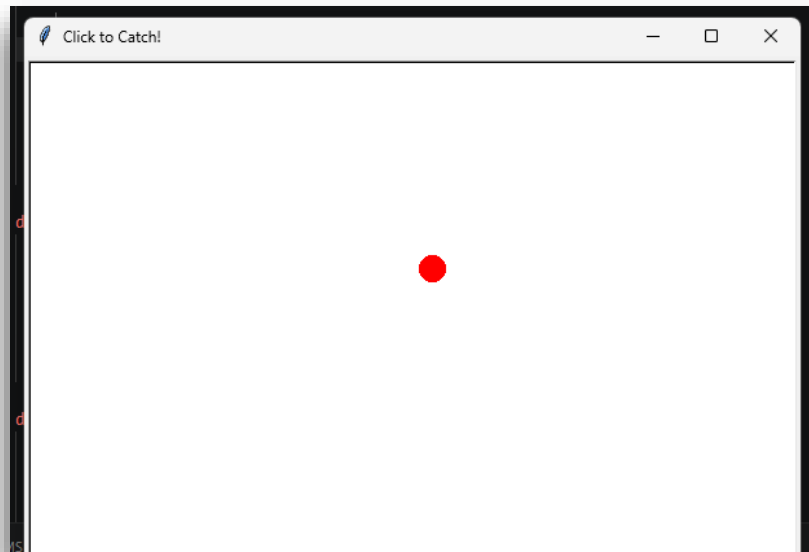
def next_round():
    global misses
    if obj.isvisible():
        obj.hideturtle()
        misses += 1
        print("Missed! Misses:", misses)
    show_object()
```

```
def catch(x, y):  
    global score  
    if obj.isvisible() and obj.distance(x, y) < 20:  
        obj.hideturtle()  
        score += 1  
        print("Caught! Score:", score)  
  
obj.onclick(catch)  
  
show_object()  
screen.mainloop()
```

Observations

- The object appears at constant intervals (1.2 seconds).
- Clicking within the object radius counts as a successful catch.
- Missing the object increments the miss counter.
- The game loop runs smoothly without freezing, thanks to ontimer scheduling.

Output/Result:



The window shows a “Red Circle” to catch it by clicking mouse on this circular object.

```
36  def catch(x, y):  
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE  
  
Game Over! Final Score: 0  
PS C:\Users\User\Desktop\CSE> & C:\Users\User\AppData\Local\Python\python  
4-64\python.exe "c:/Users/User/Desktop/CSE/Object Catch.py"  
Caught! Score: 1  
Caught! Score: 2  
Caught! Score: 3  
Caught! Score: 4  
Missed! Misses: 1  
Caught! Score: 5  
Missed! Misses: 2  
Caught! Score: 6  
Missed! Misses: 3  
Game Over! Final Score: 6  
PS C:\Users\User\Desktop\CSE> |
```

VS Code's terminal shows the output result of Caught, Misses and Final Score.